

×

[Dashboard](#)

[Inbox](#)

[Cases](#)

[My Vulnerability Reports](#)

[Report a Vulnerability](#)

[Profile](#)

[Email Preferences](#)

[Change Password](#)

[Log out](#)

VINCE

[Dashboard](#)

[Inbox](#)

[Cases](#)

[My Vulnerability Reports](#)

[Report a Vulnerability](#)

VRF#[26-05](#)-LCZZS: GitLab CI/CD OIDC id_tokens Closed

Vulnerability Report

Reported by acd421 [Andrew Dorman] on 2026-05-16, 23:34 (1 week, 1 day ago) via web submission.

ID: VRF#[26-05](#)-LCZZS

Have you attempted to contact the vendor?

Yes

Date of First Contact

2026-05-14

Summary of Previous CommunicationsHi

@acd421, Thank you for your submission. After reviewing your report, this issue has been previously reported and assessed in report [#3639503](#). Both reports describe the same OIDC implementation vulnerability where the id_token sub claim is constructed from the mutable

project_path instead of an immutable project_id. The core issue is identical: when a project is deleted, its path is immediately freed for reuse, allowing new projects created at the same path to receive OIDC tokens with identical sub claims. This enables attackers to gain access to cloud IAM roles that trusted the original project, as cloud providers accept these tokens based on the sub claim matching their trust policies. The original report was evaluated and closed as informative on April 6, 2026, indicating this behavior was assessed by the program's security team. While your report includes additional proof-of-concept automation and testing across different namespace types, the fundamental vulnerability and attack vector remain the same as previously reported. Thanks,
@h1_analyst_ace

Affected Vendor

[Hackerone.com/GitLab](https://hackerone.com/GitLab)

Amazon

Google

Microsoft

GitLab

Many More

Supply Chain Critical Risk

Affected Product

GitLab CI/CD OIDC id_tokens

Affected Version

All versions since GitLab [15.7](#) (OIDC id_tokens introduced). Tested on gitlab.com (current SaaS release)

Significant ICS/OT impact?

True

Related to AI/ML systems?

True

Reporter Name

Andrew Dorman

Reporter Email

andrew.dorman2019@gmail.com

Do you want to be acknowledged by name in any published document about this vulnerability?

YES

Do you want us to share your contact information with vendors?

YES

DescriptionVULNERABILITY: GitLab CI/CD OIDC sub claim constructed from recyclable project_path, enabling cross-cloud identity takeover

AFFECTED PRODUCT: GitLab CI/CD OIDC id_tokens (all versions since GitLab [15.7](#))

AFFECTED DOWNSTREAM PRODUCTS:

- AWS IAM OIDC Federation
(sts:AssumeRoleWithWebIdentity)
- Azure Workload Identity Federation
- GCP Workload Identity Federation

SUMMARY:

GitLab CI/CD OIDC id_tokens construct the sub (subject) claim from the mutable project_path rather than the immutable project_id. When a GitLab project is deleted, its path is immediately freed for reuse. A new project created at the same path receives OIDC tokens with an identical sub claim but a different project_id. Cloud providers (AWS, Azure, GCP) accept these tokens as the original project's identity because they can only condition trust policies on the sub and aud claims. The new project inherits full access to any cloud IAM role that trusted the deleted project.

GitLab controls the sub claim content and populates it with a recyclable identifier. GitLab's own OIDC documentation (https://docs.gitlab.com/ee/ci/cloud_services/aws/)

instructs users to configure cloud trust policies using `project_path:GROUP/PROJECT:ref_type:branch:ref:main`. Users following the recommended setup are vulnerable by default.

This is an architectural flaw in claim construction requiring coordinated remediation across GitLab and all three major cloud providers.

ROOT CAUSE:

1. GitLab places the mutable `project_path` in the OIDC sub claim instead of the immutable `project_id`
2. GitLab frees deleted project paths immediately with no cooldown period
3. [GitLab.com](https://gitlab.com) uses a single shared OIDC issuer (<https://gitlab.com>) and shared JWKS signing

keys across all 30M+ users with no tenant isolation

4. AWS, Azure, and GCP trust policies can only condition on sub and aud claims, not on custom claims like project_id

5. GitLab's own documentation recommends path-based trust policy configuration

The immutable project_id IS present in the token as a separate claim, but no major cloud provider supports filtering on it.

PROOF OF CONCEPT:

Fully automated, tested across two namespace types on infrastructure I own.

User namespace test:

- Created project at path ACD421/oidc-poc-victim (project_id: [82246357](#))

- Pushed .gitlab-ci.yml with id_tokens minting OIDC token
- Pipeline ran, AWS STS AssumeRoleWithWebIdentity succeeded, S3 bucket exfiltrated
- Deleted project via API. Path immediately freed (HTTP 404 confirmed)
- Created NEW project at SAME path (project_id: [82246363](#))
- Pushed identical .gitlab-ci.yml. Pipeline ran
- New OIDC token had DIFFERENT project_id but IDENTICAL sub claim
- AWS STS accepted the token. Same IAM role assumed. Same S3 secrets exfiltrated

Group namespace test:

- Created group oidc-poc-grp, project oidc-poc-victim (project_id: [82246947](#))
- Same test. Deleted and recreated (project_id: [82246953](#))

- Identical sub claim. AWS STS accepted. Same role, same secrets

Four different GitLab projects across two namespace types produced tokens with identical sub claims. All four successfully assumed the same AWS IAM role and exfiltrated data from the same S3 bucket.

OIDC claim comparison (user namespace):

Original (project_id [82246357](#)):

sub: project_path:ACD421/oidc-poc-

victim:ref_type:branch:ref:main

iss: <https://gitlab.com>

jti: 2eb7bc1e-ebe4-[424f-a8](#)fa-802ab8feca4c

Recreated (project_id [82246363](#)):

sub: project_path:ACD421/oidc-poc-

victim:ref_type:branch:ref:main

iss: <https://gitlab.com>

jti: d4935dca-ea3b-4328-90a2-430c694589e0

Different project_ids, different JTIs, identical sub claims.

AWS STS output:

Phase 1: exit 0, AccessKeyId

ASIA2Y6WD4DMG5FEKGVZ

Phase 2: exit 0, AccessKeyId

ASIA2Y6WD4DMDKUNSNVX

Both exfiltrated: FLAG.txt, credentials/prod.env,
credentials/api_keys.json

All testing used accounts I own (GitLab:

ACD421, AWS: [740794032344](#)). No third-party
resources were affected.

ATTACK REQUIREMENTS:

- Free [GitLab.com](https://gitlab.com) account

- Knowledge of a deleted project's path (discoverable via public activity, cached CI configs, org restructuring announcements)
- No victim interaction required after initial OIDC trust configuration
- Fully automated, completes in under two minutes

ENTERPRISE-SCALE EXPOSURE:

When an organization deletes or transfers a GitLab group during restructuring, every project path under that group is freed simultaneously. An attacker who claims the group path can recreate each project and inherit every cloud trust relationship at once. A single group deletion can expose dozens of independent cloud access grants. Group restructuring, team dissolution, and post-acquisition cleanup are routine enterprise operations.

GOVERNMENT/CRITICAL INFRASTRUCTURE EXPOSURE:

GitLab holds FedRAMP ATO at Moderate Impact Level. GitLab Dedicated for Government deploys on AWS GovCloud with FIPS-compliant encryption. DoD uses GitLab via Iron Bank hardened containers across classifications. Federal agencies and defense contractors using GitLab CI/CD with OIDC federation to AWS GovCloud are affected if they follow GitLab's recommended trust policy configuration. Recycled namespace attacks would grant IAM credentials into GovCloud accounts managing critical infrastructure.

AI/ML ATTACK VECTOR:

GitHub Copilot coding agent and similar AI

agents (Claude via GitHub, OpenAI Codex) run autonomously inside GitHub Actions with access to OIDC token minting (id-token: write permission). An AI agent assigned to an issue can autonomously create GitLab projects, delete projects, push CI configurations, and trigger pipelines. The full OIDC exploit chain can be executed by an autonomous AI agent with no human in the loop. AI coding agents represent a new autonomous execution vector for OIDC trust exploitation.

INDUSTRY PRECEDENT:

GitHub had the identical vulnerability in GitHub Actions OIDC tokens. On April 23, 2026, GitHub announced immutable subject claims that append immutable owner and repository IDs to the sub claim (e.g. repo:octocat-123456/my-repo-456789:ref:refs/heads/main). Enforcement

begins [June 18, 2026](#) for all new repositories. GitHub's changelog explicitly acknowledges: "If a repository or organization name was recycled, a new owner could mint tokens with the same subject claim, potentially gaining unauthorized access to cloud resources that still trusted the original identity."

GitHub recognized this as a security issue and is deploying the exact remediation recommended for GitLab: immutable identifiers in the sub claim. GitLab has not.

WHY COORDINATED DISCLOSURE IS REQUIRED:

1. No single vendor owns the complete fix.

GitLab controls the token content, cloud providers control the trust policy capabilities.

2. Documentation-only remediation does not protect existing users. Established OIDC trust

policies remain vulnerable. Existing users would need to update every cloud trust policy and GitLab would need to rotate JWKS signing keys to invalidate old-format tokens.

3. GitLab closed prior reports as informative (April 6, 2026, HackerOne report #[3639503](#)) and duplicate of informative (May 16, 2026, HackerOne report #[3739444](#)) without remediation.

4. Cloud providers individually bounce the report as "not our product" despite their customers having no available mitigation within their platforms.

REMEDICATION RECOMMENDATIONS:

For GitLab:

- Include immutable project_id in the sub claim (matching GitHub's approach)
- Implement path cooldown period on project

deletion

- Rotate JWKS signing keys after format change to invalidate old-format tokens
- Issue security advisory to all OIDC users

For AWS/Azure/GCP:

- Support trust policy conditions on custom OIDC claims (e.g. project_id)
- Issue security bulletins advising customers using GitLab OIDC that path-based sub claims are recyclable
- Coordinate with GitLab on migration timeline

CVSS [3.1](#): AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:N/E:F/RL:U/RC:C (Base [9.6](#), Temporal 9.4)

DISCLOSURE STATUS:

- GitLab: HackerOne reports [#3639503](#) (closed informative April 6, 2026) and [#3739444](#) (closed duplicate May 16, 2026). No fix deployed.

- Microsoft MSRC: Submitted May 16, 2026. Pending.
- Google Cloud: Submitted May 2026. Initial response out of scope, pending reframing.
- AWS: Submission pending.

Full evidence package available: GitLab CI templates, GitHub Actions orchestration workflows, job traces, OIDC claim comparisons, AWS STS output, private repository archive.

Reporter: Andrew Dorman

Contact: andrew.dorman2019@gmail.com

Phone: [202-766-9032](tel:202-766-9032)

HackerOne: acd421

GitLab: ACD421

ExploitPREREQUISITES:

1. Free [GitLab.com](https://gitlab.com) account (no paid subscription required)

2. Knowledge of a deleted project's path

Path discovery is trivial through multiple channels:

- Public GitLab activity (commits, issues, merge requests cached by search engines)
- CI/CD configuration files referencing project paths in other repositories
- Organization restructuring announcements (blog posts, press releases, internal wikis)
- GitLab API enumeration (404 on a previously-active path confirms deletion)
- Former employees or contractors with knowledge of internal project structure
- Archived documentation, READMEs, or dependency files referencing the path

No victim interaction is required after their initial OIDC trust policy configuration.

ATTACK STEPS:

1. RECONNAISSANCE: Attacker identifies a GitLab project path that was previously used and is now deleted. Attacker confirms the path is available by checking for HTTP 404.

2. NAMESPACE CLAIM: Attacker creates a new GitLab project at the exact same path. For user namespace attacks (e.g. USERNAME/project-name), attacker needs a matching username or creates one. For group namespace attacks (e.g. GROUP/project-name), attacker creates the group and project. If the group name is also freed (organization deleted the entire group), attacker claims the group path and can recreate ALL projects under it.

3. OIDC TOKEN MINTING: Attacker pushes a .gitlab-ci.yml file with id_tokens configured:

job:

id_tokens:

OIDC_TOKEN:

aud: <https://gitlab.com>

script:

- echo \$OIDC_TOKEN

GitLab's CI/CD pipeline runs and mints an OIDC token. The token's sub claim is constructed from the project_path, which is identical to the deleted project's path. The token is validly signed by GitLab's JWKS keys.

4. CLOUD CREDENTIAL THEFT: Attacker uses the OIDC token to authenticate to the victim's cloud provider:

AWS: `aws sts assume-role-with-web-identity --role-arn <victim-role> --web-identity-token <oidc-`

token>

Azure: az login --federated-token <oidc-token>

GCP: gcloud auth login --cred-file=<oidc-token-config>

The cloud provider validates the token against GitLab's JWKS keys (valid), checks the sub claim against the trust policy (matches), and issues temporary credentials. The attacker now has the same IAM role, permissions, and resource access as the original project.

5. EXPLOITATION: Attacker uses the cloud credentials to read secrets, modify infrastructure, deploy code, exfiltrate data, or pivot laterally within the victim's cloud environment. Credentials are short-lived but can be repeatedly obtained by re-running the pipeline.

TIME TO EXPLOIT: Under two minutes from

project creation to cloud credential theft. Fully automatable.

GROUP NAMESPACE ESCALATION:

The enterprise-scale attack targets group deletion during organizational restructuring:

1. Organization deletes GitLab group "company-name" containing 50 projects
2. All 50 project paths are freed simultaneously
3. Attacker creates group "company-name" on [GitLab.com](https://gitlab.com)
4. Attacker recreates each project under the group
5. Each recreated project inherits the OIDC identity of its predecessor
6. Attacker assumes 50 separate cloud IAM roles in a single operation

Group restructuring, team dissolution, post-acquisition cleanup, and rebranding are routine enterprise operations that trigger this scenario.

AI AGENT AUTONOMOUS EXPLOITATION:

GitHub Copilot coding agent, Claude (via GitHub), and OpenAI Codex run autonomously inside GitHub Actions with id-token: write permission. The full attack chain can be encoded as a GitHub Actions workflow and triggered by an AI agent assigned to an issue.

The agent can:

- Create GitLab projects via API
- Delete GitLab projects via API
- Push .gitlab-ci.yml configurations
- Trigger pipelines
- Parse OIDC tokens
- Call cloud provider STS/auth endpoints
- Exfiltrate data

All steps are standard API calls within the capabilities of any CI/CD automation or AI coding agent. No privilege escalation or exploit code is required. The attacker is using every system exactly as designed.

DETECTION DIFFICULTY:

Exploitation is extremely difficult to detect because:

- The attacker's OIDC token is validly signed by GitLab's production JWKS keys
- The sub claim matches the victim's trust policy exactly
- Cloud provider authentication logs show a normal OIDC federation event
- No failed authentication attempts occur
- The victim receives no notification from GitLab or the cloud provider

- The only anomaly is a new GitLab project at a previously-used path, which is not a standard monitoring target
- CloudTrail/Azure Activity Log/GCP Audit Log will show the role assumption but the source identity appears legitimate

The attack is indistinguishable from authorized access in every log system.

Impact The attacker gains the full cloud identity of the deleted project. Every permission, every resource, every secret that the original project's OIDC trust relationship granted is inherited by the attacker's project. The impact scales with whatever the victim configured their IAM role to access.

PROVEN IMPACT (from PoC testing on my own infrastructure):

- Full IAM role assumption via AWS STS AssumeRoleWithWebIdentity
- Four successful credential issuances across four projects and two namespace types
- S3 bucket listing and data exfiltration (FLAG.txt, credentials/prod.env, credentials/api_keys.json)
- Repeatable on demand by re-running the pipeline

CLOUD RESOURCE ACCESS (dependent on victim's IAM role permissions):

AWS:

- S3: read/write/delete any bucket the role can access
- EC2: launch, terminate, modify instances
- RDS: access databases, export snapshots
- Secrets Manager/Parameter Store: read production secrets, API keys, database credentials

- Lambda: deploy or modify functions
- IAM: if the role has IAM permissions, create additional roles or users for persistence
- KMS: decrypt data encrypted with customer-managed keys
- CloudFormation/Terraform state: read infrastructure-as-code state files containing secrets
- ECR: push malicious container images into the victim's registry
- CodePipeline/CodeDeploy: inject code into deployment pipelines

Azure:

- Key Vault: read secrets, certificates, keys
- Storage: access blobs, tables, queues
- Compute: manage VMs, container instances
- Azure SQL/Cosmos DB: access databases
- AKS: deploy to Kubernetes clusters
- Azure DevOps: if federated, pivot into additional

CI/CD infrastructure

GCP:

- Cloud Storage: read/write buckets
- Compute Engine: manage instances
- Secret Manager: read secrets
- BigQuery: query datasets
- GKE: deploy to Kubernetes clusters
- Cloud Functions: deploy or modify serverless functions

SUPPLY CHAIN COMPROMISE:

If the victim's IAM role has permissions to push artifacts (container images, packages, binaries), the attacker can inject malicious code into the victim's software supply chain. This includes:

- Pushing trojanized Docker images to ECR/ACR/GCR
- Publishing malicious packages to private

registries

- Modifying deployment artifacts in S3/Cloud Storage
- Altering infrastructure-as-code templates (Terraform, CloudFormation, ARM)
- Injecting into CI/CD pipelines that deploy to production

A single compromised IAM role with deployment permissions turns this from credential theft into a supply chain attack affecting the victim's downstream customers.

LATERAL MOVEMENT:

- Cross-account role assumption if the victim's role has sts:AssumeRole permissions to other AWS accounts
- Access to federated services (e.g., SSO, directory services) if the role is configured for

identity propagation

- VPC access via EC2/Lambda if the role has network permissions, potentially reaching internal services not exposed to the internet
- Access to other cloud providers if the victim uses multi-cloud with cross-provider trust relationships

PERSISTENCE:

- Attacker can re-run the GitLab pipeline at any time to obtain fresh credentials
- Short-lived STS tokens are renewable indefinitely as long as the attacker controls the GitLab project
- If the IAM role permits, attacker can create backdoor IAM users, roles, or access keys for persistent access independent of the OIDC token
- Attacker can modify the victim's cloud infrastructure to maintain access even after the

OIDC issue is remediated

ENTERPRISE-SCALE IMPACT:

Group namespace attack multiplies all of the above across every project in the deleted group. An organization with 50 projects under one group, each with its own cloud trust relationship, loses 50 IAM roles simultaneously. The attacker gains independent access to 50 separate cloud permission sets, potentially across multiple cloud providers and multiple cloud accounts.

GOVERNMENT/CRITICAL INFRASTRUCTURE IMPACT:

GitLab holds FedRAMP ATO at Moderate Impact Level. GitLab Dedicated for Government deploys on AWS GovCloud. DoD uses GitLab via Iron Bank hardened containers. Federal agencies and

defense contractors using GitLab OIDC to deploy to GovCloud could have their GovCloud IAM roles assumed by an attacker with a free [GitLab.com](https://gitlab.com) account. Depending on the role permissions, this could grant access to:

- Classified or controlled unclassified information (CUI)
- Defense industrial base systems
- Critical infrastructure management planes
- Government cloud environments hosting citizen data

DATA BREACH SCOPE:

The volume of data at risk is unbounded. It depends entirely on what the victim's IAM role can access. A role scoped to a single S3 bucket exposes one bucket. A role with AdministratorAccess exposes the entire AWS account. The vulnerability does not limit the

scope of access; it inherits whatever scope the victim configured.

FINANCIAL IMPACT:

- Direct data breach costs (notification, remediation, legal)
- Cloud resource abuse (cryptomining, compute consumption)
- Regulatory penalties (GDPR, HIPAA, SOX, FedRAMP) if PII/PHI/financial data is exfiltrated
- Reputational damage
- Incident response costs across multiple cloud accounts
- Supply chain compromise remediation costs for downstream customers

UNDETECTABILITY AMPLIFIES ALL IMPACT:

All of the above can occur without the victim's

knowledge. The OIDC token is validly signed. The authentication event appears legitimate in all cloud provider logs. There is no failed login, no anomalous IP (GitLab runners use shared IP ranges), no MFA challenge, no notification. The attacker could maintain access for weeks or months before detection, if detection occurs at all.

CVSS [3.1](#): AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:N (Base [9.6](#))

Scope is Changed because the vulnerability exists in GitLab's OIDC implementation but the impact is entirely in external cloud provider environments that GitLab does not control.

DiscoveryThe vulnerability was discovered accidentally during unrelated boundary condition testing on GitHub Actions, using AI-assisted offensive security research.

METHODOLOGY:

I was testing CI/CD trust boundary conditions on GitHub Actions using Claude Code (Anthropic's AI coding agent) as my primary research tool. Claude Code operated in an autonomous loop: I provided research direction, it generated hypotheses, wrote test code, executed it, analyzed results, and proposed next steps. The research session consumed approximately 12 million tokens over 24 hours of continuous operation.

While investigating how GitHub Actions interacts with external CI/CD providers and cloud trust relationships, I identified that GitLab's OIDC implementation constructs the sub claim from the mutable `project_path`. This observation led to the hypothesis that project path recycling

would produce identical sub claims for different projects, enabling cloud identity takeover.

TOOLS USED:

1. Claude Code (Anthropic Claude Opus 4.6) - AI coding agent used for research automation, hypothesis generation, exploit development, and evidence collection. Operated via tool-call interface for token efficiency.

2. GitHub Actions - CI/CD platform used to orchestrate the proof of concept. Workflows automated the full exploit chain: GitLab project creation, deletion, recreation, pipeline triggering, and evidence comparison. The GitHub Actions run logs serve as timestamped, tamper-evident proof of exploitation.

3. [GitLab.com](https://docs.gitlab.com/api/) API - Used to create, delete, and

recreate projects programmatically. Standard REST API calls, no authentication bypass or unauthorized access.

4. GitLab CI/CD with OIDC id_tokens - Standard GitLab CI/CD configuration using the documented id_tokens feature to mint OIDC tokens.

5. AWS CLI - Used to call sts:AssumeRoleWithWebIdentity with the GitLab OIDC token, then list and exfiltrate S3 bucket contents.

6. Python (inline in CI/CD jobs) - Used for JWT decoding, claim extraction, and credential parsing within the GitLab CI job.

7. curl - Used for GitLab API calls and HTTP status verification (404 confirmation of path

availability after deletion).

DISCOVERY SEQUENCE:

1. During GitHub Actions boundary testing, I observed that CI/CD OIDC tokens across platforms (GitHub, GitLab) use path-based identifiers in the sub claim.

2. I hypothesized that if the path is mutable and recyclable, the sub claim would collide after project deletion and recreation.

3. Claude Code generated a four-phase test workflow:

Phase 1: Create project, run pipeline, capture OIDC claims, verify AWS STS works

Phase 2: Delete project, confirm path freed (HTTP 404)

Phase 3: Create new project at same path, run

pipeline, capture OIDC claims

Phase 4: Compare claims, verify AWS STS still works with new project's token

4. The test confirmed the hypothesis on the first run. `project_id` changed, sub claim did not, AWS STS accepted both tokens.

5. I expanded testing to group namespaces to verify the enterprise-scale attack scenario. Same result.

6. I subsequently discovered that GitHub had independently identified and was remediating the identical vulnerability class in their own OIDC implementation (announced April 23, 2026), confirming this is a recognized vulnerability pattern across the CI/CD industry.

AI-ASSISTED RESEARCH CONTEXT:

This discovery demonstrates that AI coding agents operating in autonomous research loops can identify novel vulnerability classes through systematic boundary testing. The AI agent did not have prior knowledge of this specific vulnerability. It discovered it through first-principles analysis of how OIDC claims are constructed, combined with automated testing of the hypothesis.

The same AI agent capabilities that enabled this defensive research could be used offensively. AI coding agents running in CI/CD environments (GitHub Copilot, Claude via GitHub, OpenAI Codex) have autonomous access to OIDC token minting, cloud authentication APIs, and project management APIs. The exploit chain requires no capabilities beyond what these agents are designed to provide.

All testing was conducted on infrastructure I own (GitLab: ACD421, AWS: [740794032344](#)). No third-party resources were accessed or affected. Is there evidence that this vulnerability is being actively exploited?

YES

Exploit References

Yes. I have successfully exploited this vulnerability four times across two GitLab namespace types (user and group), achieving full cloud credential theft and data exfiltration via AWS STS AssumeRoleWithWebIdentity. Full evidence attached to this report. Whether additional exploitation by other parties has occurred is unknown but would be undetectable through normal monitoring.

Is the vulnerability publicly known?

YES

Public References

Yes. GitHub publicly described the identical vulnerability class in their own OIDC implementation on April 23, 2026 (<https://github.blog/changelog/2026-04-23-immutabl...>) explicitly acknowledging that recycled namespace paths allow unauthorized cloud resource access via identical sub claims. The attack pattern is documented. GitLab's OIDC implementation has the same flaw and has not been remediated.

Do you plan to publicly disclose this vulnerability yourself?

YES

Disclosure Plans

I have looped in Amazon(active investigation)
MSRC(review/repro) Google(the closed as not our problem)

Gitlab head of security(crickets)

Hackerone(marked as dupe informational from months ago where GitLab dismissed it)

DC3

CISA(TOC channel email)

PCII(email)

Was considering publishing a security post ran by founder of Google AI lab who used to work at Google but risk too great until patch)

CommentsGitLab closed two HackerOne reports on this issue without remediation ([#3639503](#) informative April 6 2026, [#3739444](#) duplicate May 16 2026). Google Cloud bounced it as out of scope. MSRC submission pending. Each vendor points to the others. This is why I am requesting CERT/CC coordination. I have a full evidence package (CI templates, workflows, job traces, OIDC claim comparisons, AWS STS output, private repo archive) ready to share via VINCE. I also have an existing relationship with CISA through DC3 DIB-VDP (John Repici) and PCII submission CII-PO-NH-[00029927](#) and CII-PO-TX-[00030103](#). I am available at

[202-766-9032](tel:202-766-9032) or

andrew.dorman2019@gmail.com for any follow-up. Several tokens are in there. Just FYI. I don't mind you having them but sanitize as needed if transferring.

Files

Add File

[vrf26-05-LCZZS_gitlab-oidc-combined-](#)

[evidence.zip](#) Uploaded by acd421 on 2026-05-16

[gitlab-oidc-azure-msrc-main.zip](#) Uploaded by

acd421 on 2026-05-18

[gitlab-oidc-gcp-proof-main \(1\).zip](#) Uploaded by

acd421 on 2026-05-18

[Screenshot_20260518_190010_Chrome.jpg](#) Uplo

aded by acd421 on 2026-05-18

Activity

acd421 commented on report

Greg, wanted to pass along my thanks to the CERT/CC team. They closed VRF#[26-05](#)-LCZZS as a 'procedural weakness' just 20 minutes after I submitted the full multi-cloud PoC. Since CERT officially doesn't view this cross-tenant OIDC hijack as a vulnerability, it formally clears my coordination holds. I'll be open-sourcing my AI-automated exploit framework on [June 1](#)—timed perfectly with Tal Skverer presenting this exact same 'procedural weakness' on stage at fwd:cloudsec. I'll make sure to highlight CERT's official assessment in my publication alongside CISA's active incident tracking (INC0706620). Appreciate the quick turnaround.

2026-05-24 (16 hours ago)

a

acd421 commented on report

If an AI agent I spun up in 24 hours can autonomously compromise AWS, Azure, and GCP identities using GitLab's default configuration, what do you think adversarial APTs are doing to GovCloud right now?

2026-05-21 (3 days, 14 hours ago)

a

acd421 commented on report

Thanks for the call to Anthropic.

2026-05-21 (3 days, 22 hours ago)

a

acd421 commented on report

I'm requesting reconsideration of this closure.

This report was closed as a "procedural weakness" approximately 19 minutes after the closure notification was generated. The

submission contains 4,000+ words of technical analysis, E2E proof of exploitation across three cloud providers, OIDC claim comparisons, full CI job traces, and a CVSS [9.6](#) scoring justification. I don't believe the report was fully reviewed in that timeframe.

The characterization as "procedural weakness" implies a procedure exists that prevents this attack. I am asking CERT/CC to name that procedure specifically. There is none. The victim followed the vendor's documented configuration guidance exactly. No configuration option exists in AWS, Azure, or GCP to condition workload identity federation trust on immutable OIDC claims for traditional federated credentials. The platform lacks the feature. That is an architectural vulnerability, not a procedural weakness.

Additional context that may not have been reviewed before closure:

1. GitHub independently identified and is remediating the identical vulnerability class in their own OIDC implementation. Announced April 23, 2026, enforcement begins [June 18, 2026](#). GitHub's changelog explicitly states: recycled namespace paths allow unauthorized cloud resource access via identical sub claims. If GitHub considers this a security vulnerability requiring remediation, CERT/CC should explain why the same pattern in GitLab is merely procedural.

2. Google Cloud's Workload Identity Federation documentation explicitly warns that GitHub and GitLab OIDC subject claims may not be unique to an organization, and provides attribute conditions as a countermeasure. Google built a

mitigation because they recognized this as a vulnerability class. Azure and AWS have not.

3. CISA incident INC0706620 is open on the cross-tenant component of this finding.

4. AWS is actively investigating their scope of impact.

5. MSRC closed as "not a Microsoft issue." I am disputing that closure separately. Two independent organizations closing a multi-vendor vulnerability by pointing at each other is exactly the coordination failure CERT/CC exists to resolve.

6. The proof of concept demonstrates two different GitLab users, with different user IDs and different project IDs, obtaining the same cloud provider access tokens and exfiltrating

secrets. This is cross-user identity theft, not a configuration oversight.

7. This affects every customer of every cloud provider using workload identity federation with any CI/CD OIDC provider that recycles namespace paths. The affected population is in the millions.

I have exercised good faith disclosure through GitLab (HackerOne, dismissed), MSRC (closed), Google VRP (out of scope), AWS (investigating), CISA (incident open), and now CERT/CC (closed). If CERT/CC declines coordination, I will note that all reasonable coordination channels have been exhausted when proceeding with public disclosure.

I am one researcher. I cannot coordinate remediation across four cloud providers and

multiple CI/CD platforms alone. That is literally what CERT/CC is for.

Andrew Dorman

Hollow Point Labs

[202-766-9032](tel:202-766-9032)

2026-05-20 (4 days, 17 hours ago)

A

Alex Ruschman sent email to submitter

Hello,

Thank you for your report submission to VINCE. The issues detailed in your report do not appear to be a vulnerability, but instead describe a procedural weakness. As this is not a vulnerability, we will be closing this ticket.

Thank you.

2026-05-20 (4 days, 18 hours ago)

A

Alex Ruschman sent email to submitter

Hello,

Thank you for your report submission to VINCE. The issues detailed in your report do not appear to be a vulnerability, but instead describe a procedural weakness. As this is not a vulnerability, we will be closing this ticket.

Thank you.

2026-05-20 (4 days, 18 hours ago)

a

acd421 commented on report

CISA Incident Number: INC0706620

I cannot handle the vast amount of affected people it's in many millions of customers and I cannot quantify the number of vendors. Just confirmed Hashicorp Vault(IBM owned I believe) still working through all vendors affected need a

team as I am one researcher.

2026-05-19 (5 days, 11 hours ago)

a

acd421 commented on report

Expanding scope beyond GitLab. The vulnerability class is IdP-agnostic and affects all three major cloud providers' workload identity federation:

Affected (E2E proven):

Azure Entra ID (federated identity credentials - issuer/subject/audience string match)

GCP Workload Identity Federation (attribute conditions on assertion.sub string match)

AWS STS AssumeRoleWithWebIdentity
(Condition StringEquals on sub claim)

Likely affected (same root class, not yet tested):

Oracle Cloud Infrastructure (OIDC federation)

Alibaba Cloud (RAM role SSO with OIDC)

IBM Cloud (IAM trusted profile with federation)

HashiCorp Vault (JWT/OIDC auth bound_subject)

Kubernetes (OIDC service account token projection)

SPIFFE/SPIRE (federated trust bundles)

Any platform accepting sub claim as identity binding

Exploitation vectors (any IdP):

Namespace recycling (project/repo/org delete + re-register)

Org/team rename (path changes, old path freed)

User reassignment (admin reassigns username producing same sub)

Tenant recycling on multi-tenant IdPs (tenant deleted, name re-registered)

Domain expiry on self-hosted OIDC issuers (issuer URL domain lapses, attacker registers it)

IdP admin compromise (directly mint arbitrary sub claims)

Merger/acquisition namespace consolidation (org restructuring frees paths)

Confirmed IdPs producing recyclable sub claims:

GitLab (project_path in sub, proven E2E)

GitHub (org/repo rename frees namespace)

Bitbucket (workspace/repo rename)

Any self-hosted OIDC provider (full sub control)

Proven impact across all three clouds:

Azure: Cross-user Key Vault secret theft
(attached evidence)

GCP: STS token exchange + Service Account
impersonation (attached evidence)

AWS: AssumeRoleWithWebIdentity + S3
exfiltration (original H1 evidence)

Root cause is identical across all three: cloud
provider trusts a mutable string claim as a
stable identity binding.

Fix requires immutable entity binding at the cloud provider layer, not per-IdP patches.

2026-05-19 (5 days, 20 hours ago)

acd421 commented on report

I have submitted the full bypass repos for GCP, AWS, and Azure. Gitlab dismissed it earlier this year as informational and their head of security when messaged on LinkedIn ignored it. AWS is taking it seriously. MSRC required Azure proof which I have now provided. Same with Google. GitLab remains silent.

2026-05-18 (6 days, 15 hours ago)

acd421 uploaded document

Screenshot_20260518_190010_Chrome.jpg

2026-05-18 (6 days, 15 hours ago)

acd421 uploaded document

gitlab-oidc-gcp-proof-main (1).zip
2026-05-18 (6 days, 15 hours ago)

a

acd421 uploaded document

gitlab-oidc-azure-msrc-main.zip
2026-05-18 (6 days, 22 hours ago)

a

acd421 submitted vulnerability reporting form
2026-05-16 (1 week, 1 day ago)

Comment

Add a comment to communicate with the
reporter/coordination team

[VINCE Help](#)

Do you have any comments? Give us [feedback!](#)

[CERT](#)

© 2026 Carnegie Mellon University

[Terms of Use](#)

[Vulnerability Disclosure Policy](#)

V.3.0.41